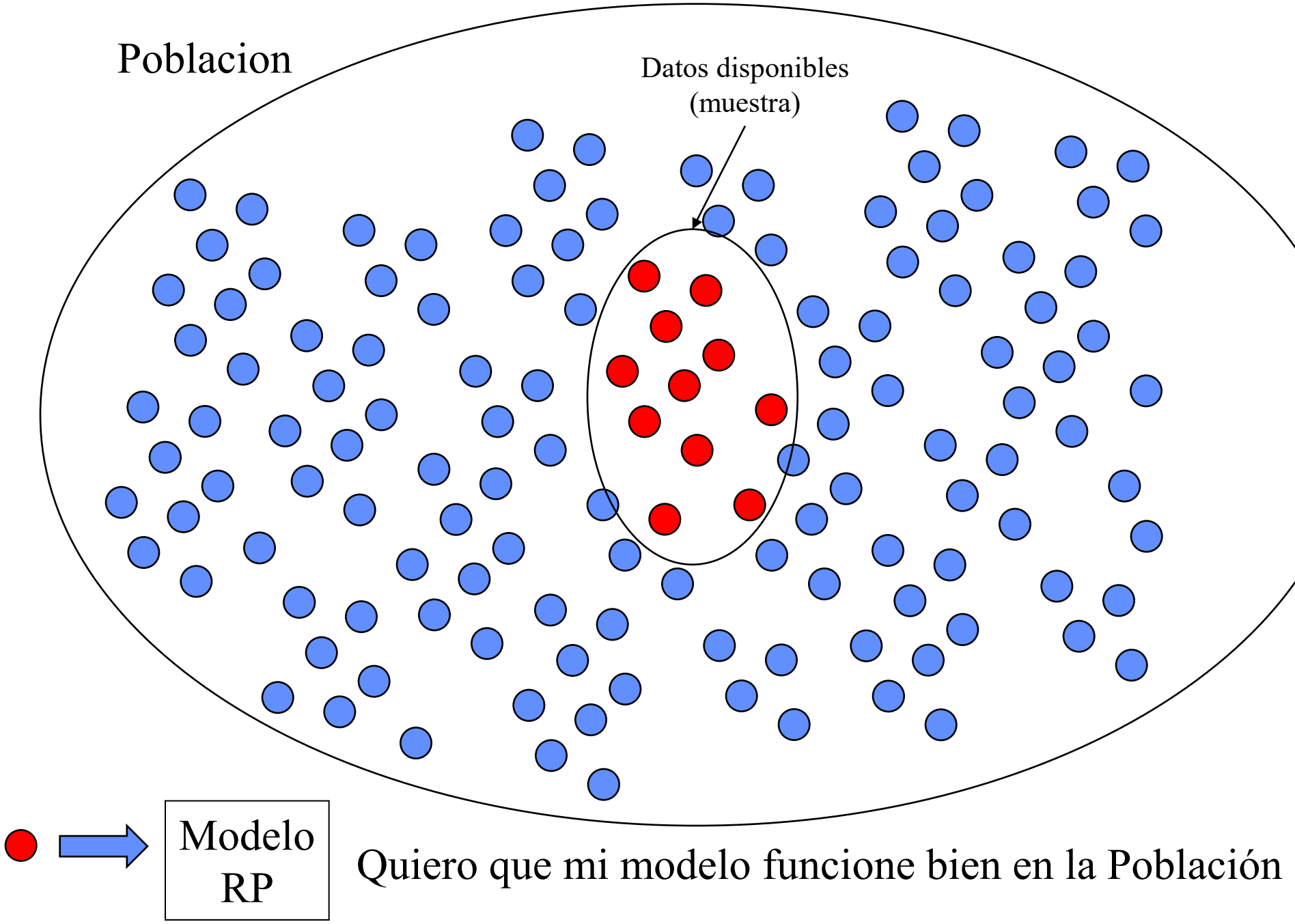
sistemas inteligentes de computación

Técnicas de Validación y Estimación del Error de Generalización

Dr. Pedro L. Galindo Riaño
Universidad de Cádiz, España


Poblacion
Datos disponibles
(muestra)
Modelo
RP
Quiero que mi modelo funcione bien en la Población

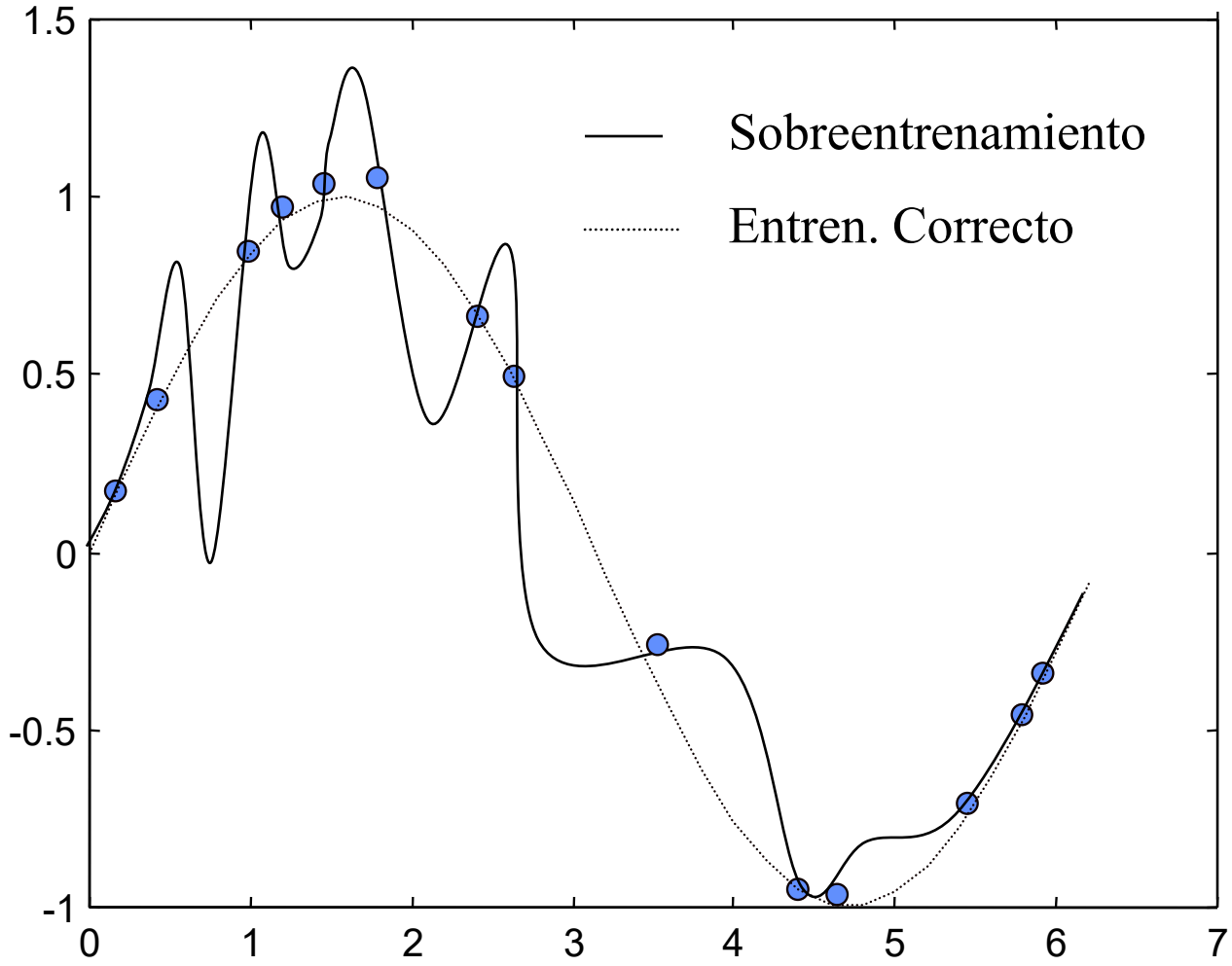
1. Introducción
¿ Por qué validar ?

- *Para probar nuestro modelo bajo **todas** las circunstancias posibles.*
- *Sin embargo, es imposible disponer de datos de entrenamiento para simular **todas** las situaciones posibles.*
- *Hemos de ser capaces de obtener un modelo capaz de **generalizar** a partir de los datos disponibles*

1. Introducción
Deficiente generalización

- *Motivada por ...*
  - *Conjunto de entrenamiento escaso*
  - *Conjunto de entrenamiento no representativo*
  - *Complejidad del modelo inadecuada*
  - *Entrenamiento inadecuado*

1. Introducción
Efecto del sobreentrenamiento



1. Introducción
Duración del entrenamiento

- *¿ Cuando parar de entrenar?*
  - *Si el error alcanza un mínimo*
  - *Si la tasa de reducción del error es inferior a un cierto valor*
  - *Tras un cierto número de "epochs"*
- *Estos métodos no garantizan una buena generalización*
- *La validación trata de subsanar este problema*

2. Métodos de estimación del error de generalización

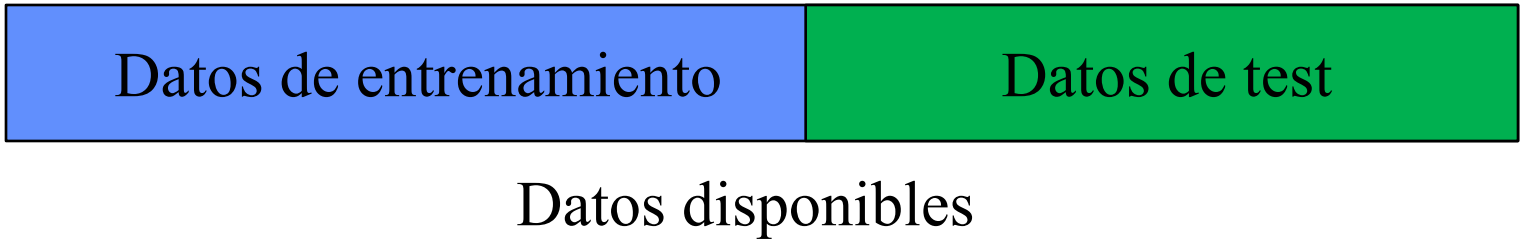
1. *Resustitución*
2. *Validación simple*
3. *Random sampling*
4. *Validación cruzada*
   a) *Orden K*
   b) *Hold-one-out (K=nº datos)*
5. *Otros*
   a) *Bootstrapping*
   b) *Stacked generalization*

3. Método de resustitución

- Consiste en detener el entrenamiento cuando el error en los datos de entrenamiento cumple una cierta condición.
- Produce sobreentrenamiento
- Se produce una medida “optimista” del error real

4. Validación simple

- *Dividir los datos en dos*
- *Una parte se utiliza para estimar el modelo*
- *La otra para calcular el error*

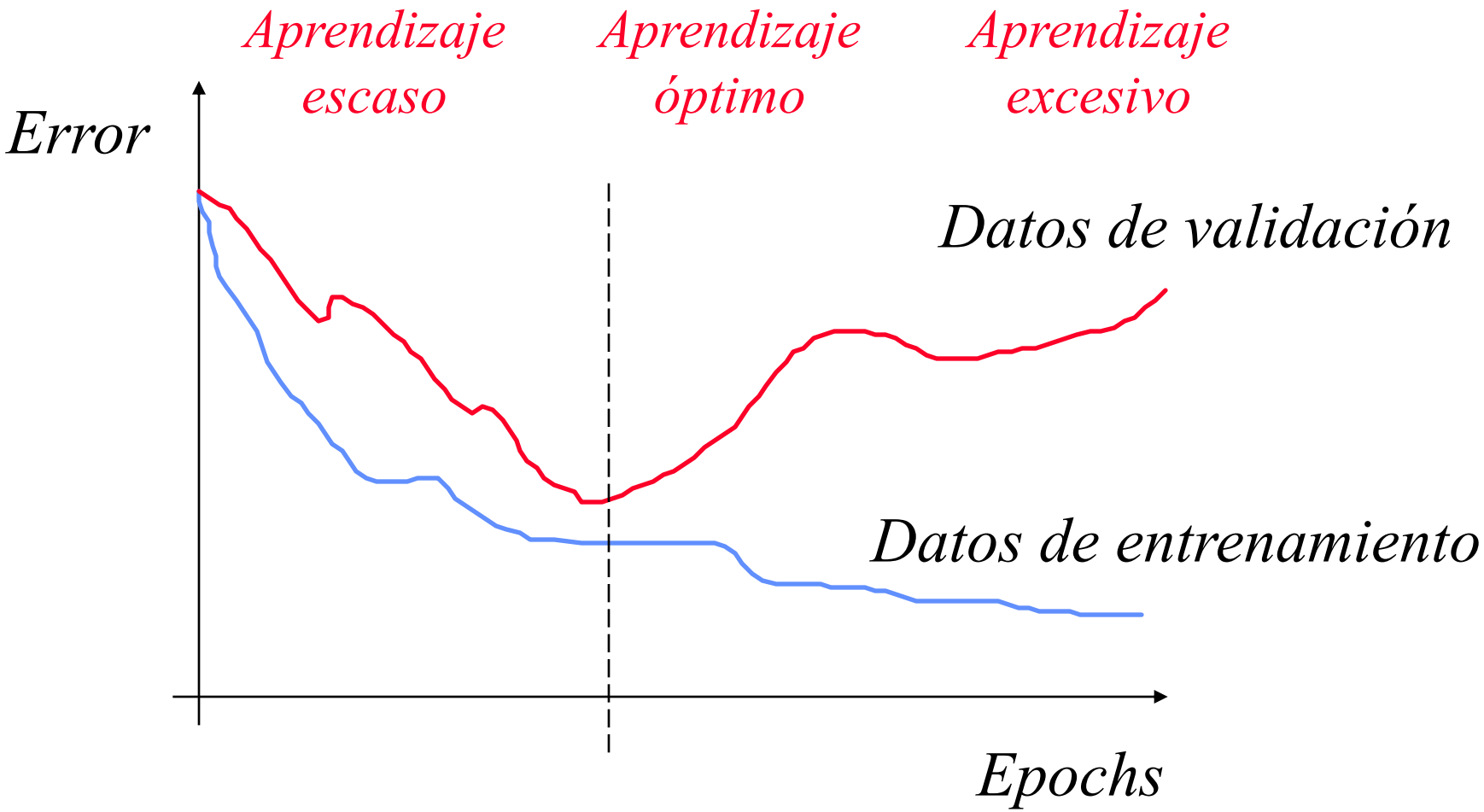


- *Se le conoce como el método "hold out"*

4. Validación simple

- *Es una parte integral del proceso de entrenamiento*
- *Divide los datos disponibles en tres grupos:*
  - *Conjunto de datos de entrenamiento (≈ 65 %)*
  - *Conjunto de datos de validación (≈ 15 %)*
  - *Conjunto de datos de test (≈ 20%)*
- *Se le conoce como el método "hold out"*

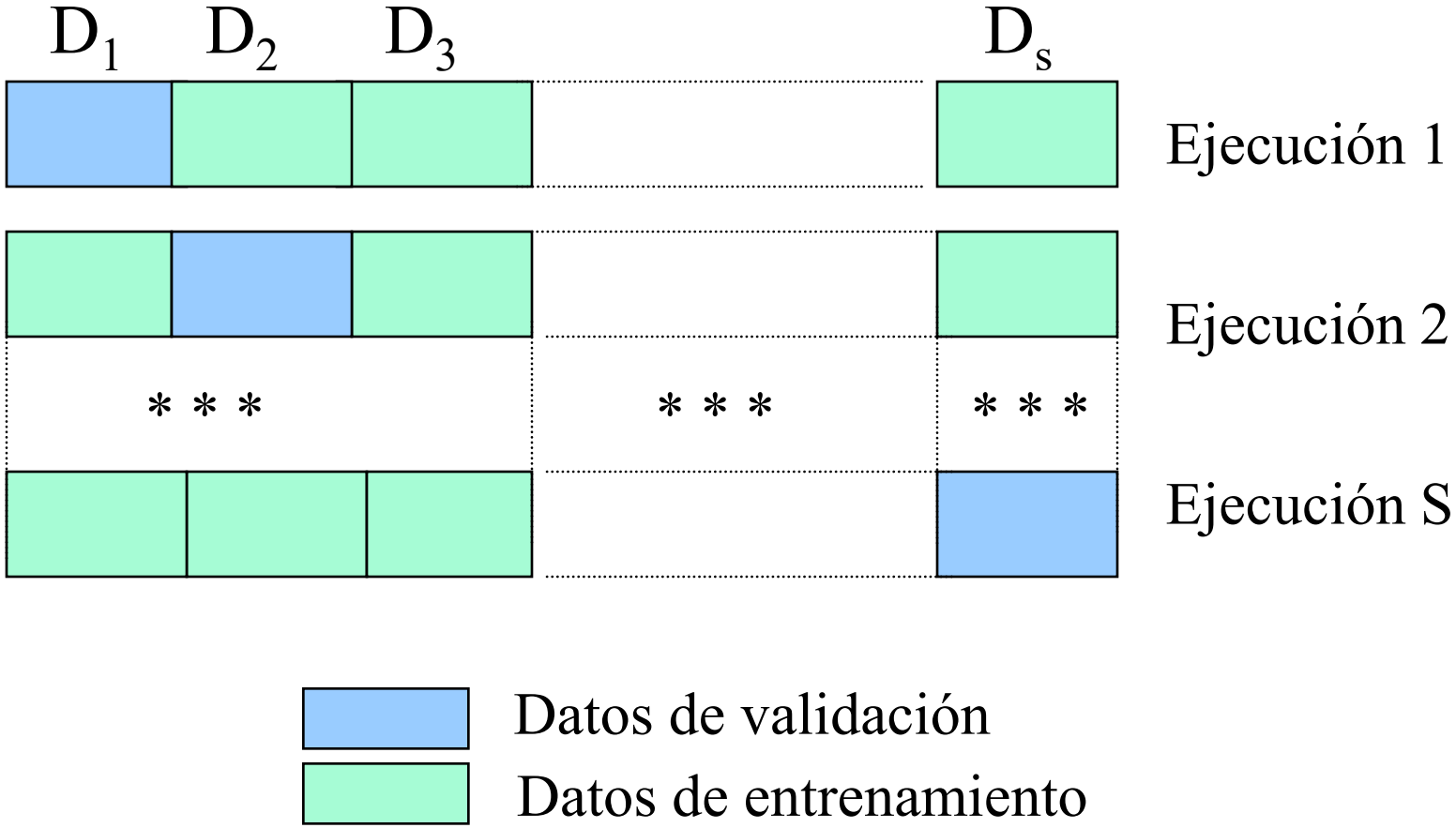
4. Validación simple
Punto óptimo de parada



4. Validación cruzada (leave-k-out/hold-k-out)

- Procedimiento
  - Divide los datos en S grupos, de tamaño similar.
  - Un valor típico de S es en torno a 10
  - Entrena el modelo S veces, dejando cada vez uno de los conjuntos para validación.
- Se puede utilizar para:
  - Calcular el error medio de generalización
  - Entrenar la red, en cuyo caso, se hace al final un entrenamiento con todos los datos disponibles, utilizando los datos de validación obtenidos para detrener el entrenamiento prematuramente

4. Validación cruzada
Representación gráfica



4. Validación cruzada
Ventajas e inconvenientes

- Ventajas
  - Es muy superior a la validación simple con conjuntos de datos pequeños.
  - Utiliza todos los datos disponibles para entrenar
- Inconvenientes
  - Precisa S+1 entrenamientos (aumento de tiempo)
  - El entrenamiento final no utiliza validación directamente

5. Validación hold-one-out (leave-one-out)

- Es un caso particular de la validación cruzada
- Consiste en la validación cruzada, considerando S=número de patrones de entrenamiento
- Bien, para funciones de error contínuas como el error cuadrático medio.

6. Bootstrapping

https://www.mbfys.ru.nl/~robvdw/DGCN22/PRACTICUM_2011/LABS_2011/ALTERNATIVE_LABS/Lesson_6.html

- Se suele utilizar si tengo **muy pocos datos**
- Consiste en imaginar que nuestra muestra es la población
- Para generar muestras diferentes, se realiza un muestreo CON REPOSICIÓN de tantos experimentos como tenga la muestra original

```
y = [10 14 23 18 12]
[m,indices] = bootstrp(100,@mean,y);
```

Concepto de hiperparámetro

- Se llaman *hiperparámetros* a aquellos que se fijan por el usuario antes de estimar los parámetros del modelo en un proceso de aprendizaje
- Definen normalmente la arquitectura de un modelo (nº capas en una red, nº nodos y profundidad en un árbol, etc) o su entrenamiento (algoritmo, nº de iteraciones, tasa de aprendizaje, parámetros de regularización, etc.)
- Se usa un modelo base, y los *hiperparámetros* definen sus posibles variantes

Fases del entrenamiento

- Se dividen los datos en DOS partes:
  - Datos de **entrenamiento**, utilizados para definir COMPLETAMENTE el modelo
  - Datos de **test**, utilizados para dar el error final del modelo
- Es frecuente hacer la fase de entrenamiento en dos fases:
  1. Determinar los **hiperparámetros** que minimizan el error estimado del modelo. Para ello, necesitemos dividir los datos de entrenamiento en dos: "**entrenamiento**" y "**validación**"
  2. Determinar los **parámetros** del modelo: una vez fijados los hiperparámetros, se calculan los parámetros internos del modelo, finalizando el proceso
- Los datos de test se usan UNA sola vez, cuando la arquitectura y parámetros del modelo están perfectamente definidos, y usando los datos de **test**

- NINGÚN DATO USADO PARA EL ENTRENAMIENTO PUEDE SER USADO EN LA FASE DE TEST